

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

LangGraph + ChromaDB + Groq LLM + HITL | 2025

1. Introduction

What is RAG? Retrieval-Augmented Generation combines LLM generation with document retrieval. Instead of relying on training data, RAG fetches relevant chunks from a trusted knowledge base at query time and provides them as grounding context to the LLM — preventing hallucination and enabling domain-specific answers.

Why RAG? Enterprise knowledge (support manuals, policies) is not in LLM training data. RAG solves this without expensive fine-tuning — knowledge base updates are instant and every answer is traceable to a source document.

Use Case: A customer support assistant for ACME Corporation that answers questions about return policies, shipping, warranties, and product troubleshooting from a support manual, while escalating billing disputes, legal queries, and urgent complaints to human agents.

2. System Architecture

The system has two pipelines sharing ChromaDB as the integration point:

- Ingestion Pipeline (offline): TXT/PDF → Chunker → HuggingFace Embeddings → ChromaDB
- Query Pipeline (online): User Query → LangGraph(intent→retriever→grader→generator) → Answer
- HITL Branch: Escalation keywords detected → hitl_node → 'Escalated to human agent'

3. Design Decisions

Decision	Choice	Reason
Chunk size	512 chars, 50 overlap	Best balance of context quality vs retrieval precision
Embedding model	all-MiniLM-L6-v2 (local)	Free, fast, 384-dim, no API cost
LLM	Groq LLaMA 3.1-8b-instant	500+ tokens/sec, free tier, low latency
Vector DB	ChromaDB (local persist)	Simple setup, Python-native, no infra needed
Workflow	LangGraph StateGraph	Stateful, conditional edges, HITL support built-in
Confidence threshold	0.70	Calibrated to balance false positives vs wrong answers

4. Workflow & Conditional Logic

Every user query flows through the LangGraph StateGraph. The **intent_node** checks for escalation keywords using string matching. If found, the conditional edge routes to **hitl_node** which returns an escalation message. Otherwise, **retriever_node** embeds the query and fetches top-k chunks from ChromaDB. **grader_node** filters the top 3 most relevant chunks. **generator_node** builds a context-grounded prompt and calls Groq LLaMA to generate the final answer.

Prompt Design:

System: "You are a support assistant. Use ONLY this context: {context}. Question: {query}" → LLM generates grounded answer only from retrieved chunks.

5. HITL Implementation

Role: HITL (Human-in-the-Loop) is a safety mechanism that prevents incorrect or harmful automated responses from reaching customers.

Trigger Condition	Action Taken
Escalation keyword in query (refund, legal, urgent, fraud...)	Route to hitl_node
No chunks retrieved from ChromaDB	Route to hitl_node

LLM confidence < 0.70	Route to hitl_node
User says 'speak to human agent'	Route to hitl_node

- Benefits: Trust, error prevention, legal compliance, audit trail
- Limitations: Higher latency (5+ min), requires human availability, higher cost

6. Testing Strategy & Results

Test	What It Tests	Result
test_config	Config class loads all env variables correctly	PASSED
test_document_loader	TXT file loads and chunks into Documents	PASSED
test_graph_state	SupportState TypedDict has correct fields	PASSED
test_routing	route_intent() returns correct node name	PASSED
test_hitl	Ticket create, fetch, resolve, list via SQLite	PASSED
test_graph_builds	build_graph() compiles without errors	PASSED

7. Challenges, Trade-offs & Future Enhancements

- Challenge: ChromaDB version conflicts — fixed by pinning chromadb==0.5.23 + langchain-chroma==0.1.4
- Challenge: LangGraph config passing — solved by wrapping all config in {'configurable': {...}}
- Trade-off: Local embeddings (free, private) vs OpenAI embeddings (higher accuracy, costly)
- Trade-off: HITL safety (slower, needs human) vs full automation (fast, risk of wrong answers)
- Future: Multi-document support, feedback loop, memory integration, FastAPI REST endpoint
- Future: Hybrid retrieval (dense + BM25), reranking, Docker deployment, Pinecone migration